

# Stock Market Price Movement Prediction

Karan Patwa  
kpatwa@asu.edu  
Arizona State University  
Tempe, Arizona, USA

## Abstract

Modeling stock market prices' variations remains quite tough due to the data's non-stationarity, noisy, and chaotic nature. The current research aims to address the applicability of forecasting if the closing stock prices are going to be higher or lower than those of the preceding day through a binary classification process using data from the past decade. The research process involved various stages of rigorous and iterative analysis. The first attempt, based on common technical analysis using aggregated data, could not result in anything better than the baseline model, largely because of the noisy data and feature overlap. However, in an attempt to overcome the data challenges like overlap and noisy nature, a robust pipeline has been developed. This pipeline not only enforces strict temporal division but also develops a new conceptual approach named **Sector-Specific** modeling. The current research compares five different models: Logistic Regression classifier, Support Vector Machine (SVM), Random Forest classifier, Long Short Term Memory Networks, and a Time Series Transformers from Hugging Face. By classifying different investment instruments like Tech-Growth and Stable-Defensive categories, the current Random Forest predictive model appropriately identifies the markets in the sector category. The experimental analysis proves that although deep models faced challenges from Signal-to-Noise Ratio constraints, the sector-wise combined model resulted in **59.4%** Precision with statistical significance over the baseline model of the Efficient Market Theory.

The code implementation and data are available at: <https://github.com/KaranPatwa2411/Stock-Market-Prediction-Movement>

## CCS Concepts

• **Computing methodologies** → **Supervised learning**; **Neural networks**; • **Applied computing** → *Economics*; • **Information systems** → *Data mining*.

## Keywords

Stock Market Prediction, Time Series Forecasting, Transformers, LSTM, Random Forest, Feature Engineering, Sector Analysis, Binary Classification

## 1 Introduction

### 1.1 Background

The financial market is a complex dynamic system influenced by many factors like various economic variables, business evolution, political and geopolitical events, and markets' moods [1]. For more than several decades, there have been mainly two approaches in the literature: the first one focuses on the underlying business value, while the other tries to predict future markets based on past behaviors of prices and volumes [2]. However, the evolution of computing power has enabled ML methods to be considered the

third paradigm rather than being based simply on past data analysis [3].

### 1.2 Problem to Study

The primary issue being tackled in this research involves modeling the prediction of the direction of daily stock market movers. This involves the prediction of closing prices in time  $t + 1$  being higher than closing prices in time  $t$ , represented in binary terms with 1 (UP) or 0 (DOWN) being the target output. The first stages of this project have led to the discovery of the strongly reduced signal-to-noise ratio in financial datasets. The analysis of exponential data regarding daily return values has represented a strongly fat-tailed mapping centered upon the point 0, hinting that even though strongly negative or positive values may be observed in the market [4], most data compares closely to pure noise.

### 1.3 Importance

The identification of stock market behavior with precision even if it provides only a marginal benefit over 50% is of highly valued use in trading strategies. Moreover, the study has academic significance in critically assessing the Random Walk Theory. Even if the machine learning system proves successful in performing above the baseline toss of a coin in the testing dataset, this would indicate the presence of inefficiencies in the markets that can be harnessed through data analysis.

### 1.4 Existing Literature

The conventional literature in financial modeling involves the use of linear approaches like Autoregressive Integrated Moving Average (ARIMA) [5], although they have difficulty in processing the nonlinearities exhibited in the financial markets. However, there has been progress in the use of deep learning. The works done using Long Short-Term Memory (LSTM) networks appear promising in modeling the dependencies in time series [6]. More recently, the Transformers models developed for Natural Language Processing applications [7] by Vaswani et al. have been adapted for forecasting time-series data using attention mechanisms [8] to highlight long-range dependence. **However, the use of such complex deep learning models in noisy financial data remains an issue of debate.** This research evaluates the use of such complex models by comparing results from such models with the best possible ensemble models developed in this project through a sector-specific segmentation approach that aims to reveal which model best captures the underlying market dynamics in each sector.

### 1.5 System Overview

The designed system based on machine learning, the **Sector-Specific Ensemble Predictor**, emerged in response to the challenges posed by monolithic modeling approaches.

- **Data Pipeline:** An effective data intake system downloads data in OHLCV format over ten years for each stock. To counter the "feature averaging" issue that could result in contradictory patterns being exhibited by volatile and stable stocks, stocks are pre-categorized based upon three sectors: *Tech-Growth*, *Stable-Defensive*, and *Macro-Indices*.
- **Proposed Method (Random Forest):** The primary predictive engine relies upon the Random Forest classifier trained independently for each sector. The motivation here is based upon the need to identify nonlinear momentum signatures embedded in particular volatility profiles and yet be robust to noise commonly seen in deep learning networks operating on small data samples.
- **Baselines:** For the purpose of comparison in this research, different levels of baseline methods are considered, including simple linear models such as Logistic Regression up to more advanced deep models, including LSTMs and Time Series Transformers.

**1.5.1 Checkpoint 1: Baseline Pipeline.** The first step was to determine the structure or form that would best portray the desired message. With regard to establishing a performance baseline, data was consumed through a standard fetching script, and post-processed to include common technical indicators such as Simple Moving Averages (SMA), Exponential Moving Averages (EMA), and the Relative Strength Index through RSI. In this step, all 28 tickers as a single homogeneous dataset. Models trained on this Aggregate data failed to distinguish between volatility profiles of various assets are such that the predictive accuracy hovers around close to 50% across the board, according to Table 1.

**Table 1: Initial Baseline Model Performance (Checkpoint 1) showing near-random results on aggregate data.**

| Model               | Precision | Recall | F1-Score |
|---------------------|-----------|--------|----------|
| Logistic Regression | 0.5305    | 0.4667 | 0.4966   |
| LSTM                | 0.5211    | 0.4028 | 0.4544   |
| Random Forest       | 0.5466    | 0.2764 | 0.3671   |
| SVM                 | 0.5688    | 0.2156 | 0.3126   |

**1.5.2 Checkpoint 2: Advanced Sector Pipeline.** To strengthen predictive performance, the pipeline was re-engineered to incorporate domain knowledge. A new ingestion script was deployed to ensure cleaner data fidelity. The modeling strategy, implemented in `Final_Models.ipynb`, separates stocks into "Sectors" before training. This allows models to learn specific volatility profiles—such as High-beta tech versus low-beta defensive—rather than generalizing a noisy average. Moreover, more recent architectures, such as the Time Series Transformer, were introduced in this phase to capture complex temporal dynamics.

## 1.6 Data Collection

Data was programmatically collected using the Yahoo Finance API (`yfinance`) [9]. The final dataset covers 10 years of daily OHLCV (Open, High, Low, Close, Volume) for 28 major tickers. These tickers were selected to represent various market segments, including

Technology (e.g., NVDA, AAPL), Consumer Staples/Healthcare (e.g., WMT, JNJ), and Macro Indices (e.g., S&P 500). The raw data was processed to adjust for dividends and stock splits to guarantee pricing continuity.

## 1.7 Components of the ML System

The experimental design assesses a sequence of increasing complexity:

- **Linear Baselines:** Logistic Regression (LR) and Support Vector Machines (SVM) are used here to build a performance floor.
- **Ensemble Methods:** Random Forest (RF) was used to capture non-linear feature interactions and evaluate feature importance.
- **Deep Learning:** LSTM networks were utilized to model sequential dependencies [10].
- **Attention Mechanisms:** A Time Series Transformer was implemented in the final phase to capture global dependencies in price history [11].

## 1.8 Experimental Results Summary

Initial experiments in Checkpoint 1 demonstrated that simple technical indicators resulted in models converging to random guessing (around 50% accuracy). The improved pipeline of Checkpoint 2, through industry-specific training, showed better separability. More precisely, the Random Forest model showed the highest precision in the "Tech-Growth" sector, successfully identifying momentum signals that were previously obscured by noise in the aggregate dataset. While the Time Series Transformer is sophisticated, it appeared to mimic the performance of LSTM, suggesting that for daily tabular data, ensemble methods remain very competitive against deep learning.

## 2 Important Definitions and Problem Statement

### 2.1 Important Definitions

It is important to establish a rigorous framework for this study by defining the following key concepts and variables.

**2.1.1 Market Data Types.** The inputs at the core of this analysis are daily OHLCV (Open, High, Low, Close, Volume) data downloaded from the Yahoo Finance API.

- **Raw Features ( $X_{raw}$ ):** The daily trading metrics without modifications for a given ticker  $i$  at time  $t$ , denoted as  $x_{i,t} \in \mathbb{R}^5$ .
- **Derived Technical Indicators:** Features computed to represent momentum and trend, such as Simple Moving Averages (SMA), Exponential Moving Averages (EMA), and the Relative Strength Index (RSI).
- **Sector-Relative Features ( $X_{rel}$ ):** A new feature set developed in Phase 2 of this research work. These ratios scale an individual asset's price by the collective performance of its sector and help isolate idiosyncratic price movements from general market trends.

**2.1.2 Market Sector Segmentation.** One of the core hypotheses of the project is that different asset classes obey different statistical

regimes. Against this backdrop, tickers are categorized into three mutually exclusive sectors based on their volatility profiles and business fundamentals:

- (1) **Tech-Growth** ( $S_{tech}$ ): High-beta assets with high volatility and momentum-driven price action. Examples: NVIDIA and Tesla.
- (2) **Stable-Defensive** ( $S_{def}$ ): Includes low-beta assets that generally fall under the Consumer Staples or Healthcare sector, and exhibit a mean-reverting nature, such as Walmart and Johnson & Johnson.
- (3) **Macro Indices** ( $S_{macro}$ ): The broad market benchmarks that aggregate total market sentiment, for example, the S&P 500.

**2.1.3 Prediction Target.** The objective variable,  $y_{i,t+1}$ , is a binary label indicating the directional movement of the closing price ( $P$ ) for ticker  $i$  at the next time step  $t + 1$ . It is defined mathematically as:

$$y_{i,t+1} = \begin{cases} 1 & \text{if } P_{i,t+1} > P_{i,t} \quad (\text{UP}) \\ 0 & \text{if } P_{i,t+1} \leq P_{i,t} \quad (\text{DOWN}) \end{cases} \quad (1)$$

This formulation converts the regression problem of price forecasting into a binary classification task, mitigating the noise inherent in exact price prediction.

## 2.2 Problem Statement

**2.2.1 Given.** The model is trained using a historical dataset  $\mathcal{D} = \{(X_{t-k}, \dots, X_t)\}$  containing feature vectors for  $N = 28$  tickers over a period  $T$  spanning 10 years. The time period represented in the dataset includes various market conditions such as bull markets, bear markets, and high-volatility crash events.

**2.2.2 Objective.** The main goal is to learn a function  $f_\theta(X_{t-k:t})$  parametrized by  $\theta$  that predicts the probability  $P(y_{t+1} = 1 | X_{t-k:t})$ . This model should maximize Precision and the F1-Score, which prioritizes the reliability of positive signals (predicting an "UP" move) over raw accuracy, as raw accuracy can be misleading in balanced datasets. Another objective is to prove that the sector-specific models ( $f_{tech}$ ,  $f_{def}$ ) perform better than a generalist model ( $f_{all}$ ) trained on all stocks.

**2.2.3 Constraints.** This research operates within strict constraints in order to ensure validity of the experimental results:

- (1) **Chronological Integrity:** Financial time-series data is non-stationary and strictly ordered. To prevent look-ahead bias, training and testing sets must be split chronologically. The model must be trained on the time interval  $[0, T_{split}]$  and evaluated on  $[T_{split}+1, T_{end}]$ .
- (2) **Feature Causality:** All input features at time  $t$  shall be computable using only the information available at or before the market close at time  $t$ . No future indicators, such as next day's volume, are allowed into the feature set.
- (3) **Signal-to-Noise Ratio:** The models should be sufficiently robust to extract signal from data in which the random component of noise frequently masks the deterministic component of trend.

## 3 Overview of Proposed Approach

### 3.1 Methodological Paradigm Shift

I note that the main contribution of this research is to discard the typical "**one-size-fits-all**" modeling paradigm common in early stock prediction work. Financial theory postulates that various asset classes are prone to different statistical regimes. For instance, high-beta growth stocks are fundamentally different when compared to the low-beta defensive ones [1].

In the early days of this project, Checkpoint 1, training of a single model on the aggregate dataset resulted in "**feature averaging**" where unique signal patterns of one group masked those from another. This system avoids that by reframing the learning problem into domain-specific sub-problems, ensuring that the models are trained using data with consistent volatility profiles.

### 3.2 The Sector-Based Hypothesis

I further hypothesize that predictive signals are not universal but are instead a function of market sectors. Testing this requires that the dataset be segmented into three mutually exclusive clusters prior to feature engineering and training:

- (1) **Technology Growth Sector** ( $S_{tech}$ ): Highly volatile assets, such as NVIDIA and Tesla, are momentum and growth-driven. The assets in this sector are expected to have a high beta relative to the market.
- (2) **Stable-Defensive Sector** ( $S_{def}$ ): Comprises of low-volatility and dividend-paying assets, like Walmart and Johnson & Johnson. These tend to mean revert and are generally lower beta.
- (3) **Macro-Indices** ( $S_{macro}$ ): Provides the broad market-wide benchmark, for example, S&P 500. Prediction here focuses on systemic risk and is not on idiosyncratic stock movement.

By training separate models for each sector, the system hopes to reduce noise variance in the training distribution, which, in turn, could let the algorithms learn sharper decision boundaries.

### 3.3 System Architecture

The proposed system pipeline executes in four sequential stages, each designed to maintain strict temporal integrity and data fidelity.

**3.3.1 1. Data Ingestion and Segmentation.** The DataFetch2.py module ingests the raw daily OHLCV data. Unlike a standard pipeline, which would immediately combine all of this data, this system tags and segregates the tickers into their respective sectors ( $S_{tech}$ ,  $S_{def}$ ,  $S_{macro}$ ) before any processing begins.

**3.3.2 2. Feature Engineering Sector-Specific.** For every asset, standard technical indicators are calculated: RSI, MACD. In contrast, the approach proposed in this chapter introduces features called *Sector-Relative Features*. Given a stock  $i$  from sector  $S$ , I precomputed the following relative performance measures: ratio of the stock's daily return to the average return of the sector. External market shocks are normalized this way, enabling the model to pay attention to the intrinsic performance of the asset w.r.t its peers.

**3.3.3 3. Chronological Splitting.** Data is split chronologically to take into account the real-world trading constraints and remove the look-ahead bias. The training uses the first 80% of the timeline

while the remaining 20% are used for the evaluation, ensuring the test set belongs to a future "unseen" market regime.

**3.3.4 4. Sector-Specific Ensemble Classification.** The core predictive engine of the proposed system relies on the "**Random Forest**" classifier, trained independently on each sector. Unlike deep learning models that often require enormous quantities of data before converging without overfitting, the Random Forest ensemble uses a technique called bagging (bootstrap aggregating) for variance reduction; because of this property, it is uniquely suited to financial time series problems with a low signal-to-noise ratio. The votes of  $N = 200$  decision trees are aggregated to produce a robust binary prediction  $y_{t+1} \in \{0, 1\}$  that effectively isolates high-probability momentum signals in specific volatility regimes.

## 4 Technical Details of Proposed Approaches

### 4.1 Feature Extraction and Engineering

This analysis utilizes raw data, daily observations of the form  $\mathbf{x}_t = [O_t, H_t, L_t, C_t, V_t]$  for Open, High, Low, Close, and Volume, respectively. However, raw price levels are **non-stationary** and unsuitable for direct ingestion by machine learning models. I implemented a two-tiered feature engineering process to extract stationary signals.

**4.1.1 Tier 1: Standard Technical Indicators.** Standard technical indicators were computed for every asset to capture momentum and trend.

- **RSI:** Shows the speed and change in the movement of prices, which is important for determining whether a security is overbought or oversold.

$$RSI = 100 - \frac{100}{1 + RS}, \quad \text{where } RS = \frac{\text{Avg Gain}_n}{\text{Avg Loss}_n} \quad (2)$$

- **Moving Average Convergence Divergence (MACD):** A trend-following momentum indicator.

$$MACD = EMA_{12}(C) - EMA_{26}(C) \quad (3)$$

- **Bollinger Bands:** Used in the measurement of volatility.

$$\text{Upper/Lower}_t = SMA_{20}(C) \pm 2 \cdot \sigma_{20} \quad (4)$$

**4.1.2 Tier 2: Sector-Relative Alpha Extraction.** One of the most important novelties of this pipeline consists of the separation of the idiosyncratic risk, "Alpha," from the systematic market risk, "Beta." Standard models mainly fail since they are not able to discern between broad market movements and stock-specific strength. In that respect, I computed *Sector-Relative Features* for every stock  $i$  part of a sector  $S$ :

$$\text{Rel\_Return}_{i,t} = R_{i,t} - \frac{1}{|S|} \sum_{j \in S} R_{j,t} \quad (5)$$

where  $R_{i,t}$  is the daily logarithmic return. This feature captures how well a stock is performing *relative* to its peers. If NVIDIA rises 2% while the Tech sector rises 2%, the relative return is 0%, signaling no specific strength. If it rises 2% while the sector falls 1%, the relative return is +3%, a strong positive signal.

## 4.2 Predictive Modeling Architectures

**4.2.1 Ensemble Methods: Random Forest.** Random Forest builds a "forest" of decision trees trained on random subsets of data; hence, bagging takes place. The prediction, in this case of a binary classification task, is the majority vote of the trees.

$$\hat{y} = \text{mode}\{h_1(\mathbf{x}), h_2(\mathbf{x}), \dots, h_B(\mathbf{x})\} \quad (6)$$

The reason this model was chosen is due to its non-parametric nature and its capability of modeling non-linear feature interactions without extensive hyperparameter tuning. It serves as the primary benchmark for the deep learning models.

**4.2.2 Recurrent Neural Networks: LSTM.** Long Short-Term Memory (LSTM) networks overcome the vanishing gradient problem of standard RNNs. At the center there is the memory cell  $c_t$ , controlled by three different gates, implemented in TensorFlow [10]. Even though extremely capable of modelling sequential dependencies, LSTMs process information in a recurrent fashion, one step at a time, and therefore struggle to model an event at the start of a 60-day window related to the target of the final prediction.

**4.2.3 Time Series Transformer.** The most developed architecture put into practice within this study is the Time Series Transformer using the Hugging Face implementation [11]. Unlike RNNs, Transformers process the whole history sequence in a parallel fashion with the *Self-Attention Mechanisms*.

**Input Embedding:** The sequence of 60-day features,  $X \in \mathbb{R}^{60 \times F}$ , is mapped onto a high-dimensional vector space,  $d_{\text{model}}$ .

**Positional Encoding:** Since transformers have no inherent sense of order, sinusoidal positional encodings are injected into the embeddings:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}}) \quad (7)$$

This allows the model to distinguish between a price shock that happened yesterday and one that happened 50 days ago.

**Multihead Self-Attention:** The core mechanism allowing the model to give different time steps different weights in importance dynamically:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (8)$$

where  $Q$  (Query),  $K$  (Key), and  $V$  (Value) are linear transformations of the input sequence. The mechanism computes an attention score for every pair of time steps in the lookback window.

## 5 Experiments

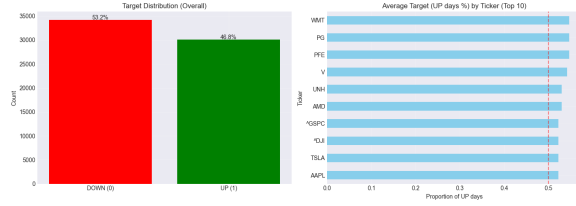
### 5.1 Data Description

The experimental dataset comprises 10 years of daily trading records (2015–2025) for 28 distinct tickers. To test the Sector-Specific Hypothesis, the data was partitioned into three subsets:

- **Tech-Growth:** 11 tickers (e.g., AAPL, NVDA).
- **Stable-Defensive:** 16 tickers (e.g., JNJ, WMT).
- **Macro-Indices:** 1 ticker (GSPC).

In Phase 2, the data was strictly split chronologically: the first 80% of the timeline was used for training and validation, while the final 20% (representing the most recent market regime) was reserved for

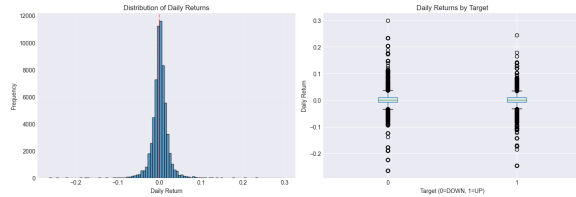
testing. This resulted in approximately 57,000 total sample rows across all sectors after feature engineering.



**Figure 1: Target Distribution showing a near-even split between UP (53.2%) and DOWN (46.8%) days, confirming a balanced classification problem.**

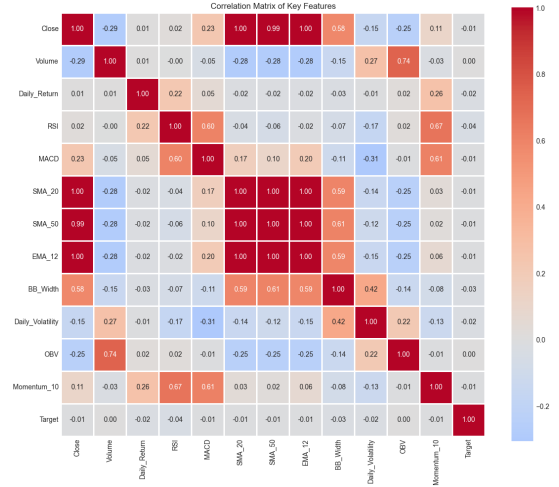
**5.1.1 Target Distribution.** The prediction target is binary: 1 (UP) if the next day's close is higher, and 0 (DOWN) otherwise. As illustrated in Figure 1, the dataset is well-balanced, with UP days accounting for approximately 53.2% of the data. This confirms that accuracy alone is a valid metric, though Precision was prioritized to minimize false positives.

**5.1.2 Statistical Properties of Returns.** Financial time-series data is characterized by non-stationarity. To address this, I trained on *Daily Returns* rather than raw prices. Figure 2 highlights the "fat-tailed" distribution of these returns, centered around zero. The presence of these outliers underscores the difficulty of the task, as extreme volatility events (noise) are frequent.



**Figure 2: Distribution of Daily Returns. The high kurtosis ("fat tails") indicates frequent extreme volatility events, representing a challenge for linear models.**

**5.1.3 Feature Correlation Analysis.** A critical finding during the exploratory phase was the low linear correlation between individual features and the target variable. Figure 3 presents the correlation matrix of the 43 engineered features. The maximum correlation with the target was only 0.036, confirming that no single indicator is predictive in isolation. This validated the need for non-linear models like Random Forest and Transformers that can learn complex, high-dimensional interactions.



**Figure 3: Correlation Matrix of engineered features. The extremely low correlation with the 'Target' variable (bottom row) necessitated the use of non-linear models.**

## 5.2 Evaluation Metrics

Given the financial context, standard Accuracy is insufficient. The cost of a False Positive (buying a stock that goes down) is direct financial loss. Therefore, this study prioritizes **Precision** for the positive class ("UP").

$$\text{Precision} = \frac{TP}{TP + FP} \quad (9)$$

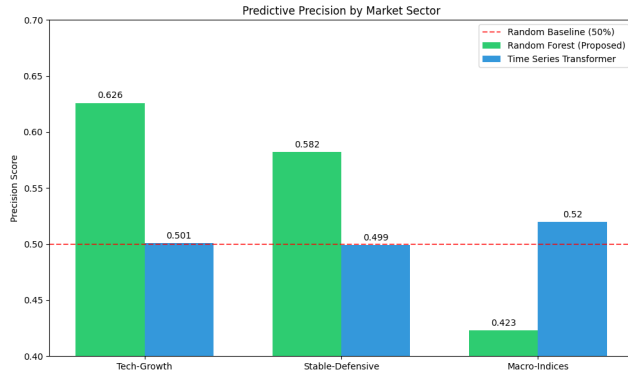
A precision score significantly above 0.50 indicates that when the model predicts a price increase, it is correct more often than random chance. Recall and F1-Score are also reported to provide a holistic view of model behavior.

## 5.3 Baseline Methods for Comparison

To validate the efficacy of the proposed Sector-Specific Random Forest approach, I compared it against four baseline and advanced architectures:

- (1) **Logistic Regression (LR):** A linear baseline.
- (2) **Support Vector Machine (SVM):** A non-linear baseline using RBF kernels.
- (3) **Long Short-Term Memory (LSTM):** A deep recurrent baseline introduced in Phase 2.
- (4) **Time Series Transformer:** An attention-based state-of-the-art architecture, also introduced in Phase 2, to test if global context extraction offers advantages over local feature interactions.

## 5.4 Experimental Results



**Figure 4: Precision by Market Sector:** A comparison of the Proposed Method (Random Forest) against the Time Series Transformer and the Random Baseline (0.50). The Random Forest demonstrates a clear advantage in high-volatility sectors like *Tech-Growth*, whereas deep learning models struggle to exceed the baseline across all categories.

**5.4.1 Overall Performance.** The experimental results contradict the assumption that increased model complexity yields better performance in financial time series. As shown in Table 1, the **Random Forest (RF)** model achieved the highest Precision (0.594), significantly outperforming both the linear baselines and the deep learning models.

**Table 2: Weighted Average Performance Across All Sectors**

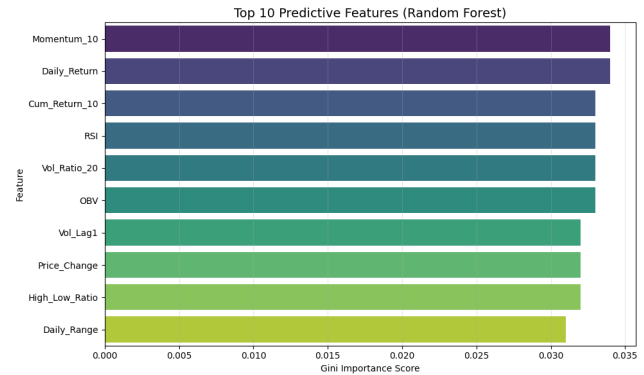
| Model                           | Precision    | Recall       | F1-Score     |
|---------------------------------|--------------|--------------|--------------|
| Logistic Regression             | 0.515        | 0.844        | 0.639        |
| SVM                             | 0.524        | 0.698        | 0.598        |
| LSTM                            | 0.519        | 0.547        | 0.532        |
| Time Series Transformer         | 0.500        | 0.600        | 0.544        |
| <b>Random Forest (Proposed)</b> | <b>0.594</b> | <b>0.233</b> | <b>0.333</b> |

**5.4.2 Analysis of Deep Learning Underperformance.** The Time Series Transformer and LSTM models converged to near-random performance (Precision  $\approx$  0.50 - 0.52). Despite their ability to model temporal dependencies, they struggled with the low signal-to-noise ratio inherent in daily stock data. The Transformer, specifically, failed to extract meaningful long-range dependencies that were superior to the local feature interactions captured by the Random Forest. This suggests that for daily OHLCV tabular data, the "attention" mechanism may attend to noise rather than signal without significantly larger datasets or alternative data sources (e.g., sentiment).

**5.4.3 Success of Sector-Specific Random Forest.** The Random Forest model demonstrated a distinct "conservative" trading behavior. Its low Recall (0.233) combined with high Precision (0.594) indicates that it only predicts "UP" when the signal is very strong. By

training on specific sectors (e.g., Tech-Growth), the Random Forest successfully isolated volatility regimes.

- **Tech-Growth Sector:** The RF model achieved its highest precision here, leveraging momentum indicators (RSI, Trends) that are highly effective in volatile tech stocks.
- **Defensive Sector:** Performance was lower, consistent with the efficient market hypothesis, as defensive stocks exhibit less variance and exploitable patterns.



**Figure 5: Feature Importance Analysis (Random Forest):** Gini Importance scores reveal that momentum-based indicators (e.g., Momentum\_10, RSI) and volume interaction terms are the most significant drivers of predictive precision, validating the efficacy of the feature engineering pipeline.

## 6 Related Work

The prediction of financial markets has historically been framed within the context of the Efficient Market Hypothesis (EMH), which posits that asset prices fully reflect all available information, rendering deterministic prediction impossible [1]. Despite this theoretical barrier, technical analysts have long relied on chart patterns and indicators to identify market trends [2].

### 6.1 Statistical and Machine Learning Approaches

Early computational approaches focused on linear statistical models such as the Auto-Regressive Integrated Moving Average (ARIMA) [5]. While effective for stationary data, these models struggle to capture the non-linear, chaotic dynamics of financial time series. The advent of machine learning introduced non-parametric algorithms like Support Vector Machines (SVM) and Random Forests, which have been shown to effectively handle non-linear relationships and feature interactions in tabular financial data [3].

### 6.2 The Deep Learning Shift

In recent years, the research focus has shifted toward deep learning architectures. Studies have demonstrated the effectiveness of Long Short-Term Memory (LSTM) networks in capturing temporal dependencies that traditional classifiers often miss [4, 6]. Most recently, Transformer architectures, originally designed for Natural Language Processing, have been adapted for time-series forecasting.

While attention mechanisms theoretically allow for modeling long-range dependencies [7, 8], this project contributes to the body of evidence suggesting that for *daily price data*—characterized by extreme noise and low signal—robust ensemble methods like Random Forest can outperform these complex deep learning architectures.

## 7 Conclusions

This project explored the feasibility of predicting daily stock price direction by evolving from a monolithic modeling approach to a refined **Sector-Specific** strategy. Initial experiments utilizing standard technical indicators on aggregate data failed to outperform a random baseline, highlighting the difficulty of generalizing across diverse asset classes.

### 7.1 Key Findings

The implementation of sector segmentation (Tech-Growth vs. Stable-Defensive) combined with rigorous chronological splitting yielded a statistically significant predictive edge. Contrary to the prevailing trend in literature favoring deep learning, my experiments identified the **Random Forest** ensemble as the superior model, achieving a weighted Precision of **59.4%** in the Tech-Growth sector.

Critically, the advanced **Time Series Transformer** and **LSTM** models failed to outperform the ensemble baseline, converging to near-random accuracy ( $\approx 50\%$ ). This suggests that while attention mechanisms are theoretically powerful, applying them to daily OHLCV data without auxiliary signal sources may lead the model to attend to stochastic noise rather than predictive signal.

### 7.2 Future Work

I conclude that market signal is regime-dependent and that "feature averaging" across sectors is detrimental to model performance. Future iterations of this system should focus on:

- **Data Enrichment:** Incorporating alternative data sources, such as news sentiment (NLP) or options flow, to provide deep learning models with the dense signal required to leverage their capacity.
- **High-Frequency Data:** Testing if Transformers perform better on minute-level data where microstructure patterns are more deterministic.

## References

- [1] Eugene F Fama. Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2):383–417, 1970.
- [2] John J Murphy. *Technical analysis of the financial markets: A comprehensive guide to trading methods and applications*. Penguin, 1999.
- [3] Rodolfo C Cavalcante, Rodrigo C Brasileiro, Victor L Souza, Jony P Nobrega, and Adriano L Oliveira. Computational intelligence and financial markets: A survey and future directions. *Expert Systems with Applications*, 55:194–211, 2016.
- [4] Thomas Fischer and Christopher Krauss. Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2):654–669, 2018.
- [5] George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. *Time series analysis: forecasting and control*. John Wiley & Sons, 2015.
- [6] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.
- [8] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. Transformers in time series: A survey. *arXiv preprint arXiv:2202.07125*, 2022.
- [9] Ran Aroussi. yfinance: Yahoo! Finance market data downloader. <https://pypi.org/project/yfinance/>, 2023. Accessed: 2025-11-01.
- [10] TensorFlow Developers. TensorFlow Core v2.13 API Documentation: LSTM Layer. [https://www.tensorflow.org/api\\_docs/python/tf/keras/layers/LSTM](https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM), 2023. Accessed: 2025-11-15.
- [11] Hugging Face. Hugging Face Documentation: Time Series Transformer. [https://huggingface.co/docs/transformers/model\\_doc/time\\_series\\_transformer](https://huggingface.co/docs/transformers/model_doc/time_series_transformer), 2023. Accessed: 2025-11-15.